Программирование в командном процессоре ОС UNIX. Ветвления и циклы

Лабораторная работа № 13 (зачётная книжка № 1132259051), РУДН

Чжан Вэньцзюнь

2026-10-20

Содержание

1 Цель работы **5**

2 Задание **6**

3 Теоретическое введение **7**

4 Выполнение лабораторной работы **9**
4.1 1. Командный файл с getopts и grep 9
4.2 2. Программа на C и анализ кода возврата 11
4.3 3. Создание и удаление N файлов 13
4.4 4. Упаковка файлов каталога в архив 14

5 Выводы **16**

6 Ответы на контрольные вопросы **17**
6.1 1. Предназначение команды getopts 17
6.2 2. Какое отношение метасимволы имеют к генерации имён файлов? 17
6.3 3. Какие операторы управления действиями вы знаете? 18
6.4 4. Какие операторы используются для прерывания цикла? 18
6.5 5. Для чего нужны команды false и true? 18
6.6 6. Что означает строка if test -f man$s$/i.$s? 19
6.7 7. Объясните различия между конструкциями while и until 19

Список литературы **20**

Список иллюстраций

4.1 Работа командного файла findgrep.sh 11
4.2 Программа sign.c, командный файл checknum.sh и анализ кода возврата 13

Список таблиц

1 Цель работы

Изучить основы программирования в оболочке ОС UNIX. Научиться писать более сложные командные файлы с использованием логических управляющих конструкций и циклов.

2 Задание

В соответствии с п. 11.2 методических указаний необходимо:

1. Используя команды `getopts` и `grep`, написать командный файл, анализирующий командную строку с ключами: `-i inputfile` (читать данные из файла), `-o outputfile` (вывести данные в файл), `-p шаблон` (шаблон для поиска), `-C` (различать большие и малые буквы), `-n` (выдавать номера строк); затем искать в файле строки по шаблону `-p`.
2. Написать на языке C программу, которая вводит число и определяет, больше оно нуля, меньше нуля или равно нулю, а затем завершается с помощью `exit(n)`, передавая код в оболочку; командный файл должен вызывать программу и по `$?` выдать сообщение о введённом числе.
3. Написать командный файл, создающий указанное число файлов, пронумерованных от 1 до `N` (`1.tmp`, `2.tmp`, ...), и умеющий удалять созданные файлы.
4. Написать командный файл, который с помощью `tar` упаковывает в архив все файлы указанной директории; модифицировать его так, чтобы упаковывались только файлы, изменённые менее недели назад (использовать `find`).

3 Теоретическое введение

Командная оболочка bash предоставляет управляющие конструкции для ветвления и циклов [1]:

- `if список-команд1; then список-команд2; [elif …; else …;] fi` — выполнение блока при истинности (код возврата 0) условия;
- `for имя in список; do … done` — перебор элементов списка (или `for ((i=0; i<n; i++))` — C-подобный цикл);
- `while список-команд; do … done` — цикл с предусловием;
- `until список-команд; do … done` — цикл с постусловием (выполняется, пока условие ложно);
- `case слово in шаблон) команды;; … esac` — выбор по шаблону;
- `break` — прерывание цикла, `continue` — переход к следующей итерации.

Функция/команда возвращает в `$?` код завершения: 0 — успех («истина»), ненулевое значение — ошибка («ложь»). Команды `true` и `false` — встроенные команды, всегда возвращающие 0 и 1 соответственно; они удобны для организации бесконечных циклов (`while true; do … done`).

Команда `getopts` разбирает ключи командной строки: `while getopts "i:o:p:Cn" opt; do case $opt in … esac; done`; двоеточие после буквы означает, что ключ имеет аргумент, доступный в `$OPTARG`; `$OPTIND` — индекс следующего аргумента.

Метасимволы (`*`, `?`, `[...]`) используются для генерации имён файлов (шаблонов): оболочка подставляет список имён, соответствующих шаблону, перед выполнением

команды.

4 Выполнение лабораторной работы

Работа выполнена в виртуальной машине VirtualBox (Fedora Linux 44, `bash 5.3.9`, `gcc 15`); командные файлы созданы в каталоге `~/work/os/lab11`.

4.1 1. Командный файл с getopts и grep

Листинг `findgrep.sh`:

```
#!/bin/bash
# поиск строк по шаблону: -i файл, -o файл, -p шаблон, -C (учитывать регистр), -n (но
IN=""; OUT=""; PAT=""; SENS=0; NUMS=0
while getopts "i:o:p:Cnh" opt; do
  case $opt in
    i) IN="$OPTARG" ;;
    o) OUT="$OPTARG" ;;
    p) PAT="$OPTARG" ;;
    C) SENS=1 ;;
    n) NUMS=1 ;;
    *) echo "usage: $0 -i файл -p шаблон [-o файл] [-C] [-n]" >&2; exit 1 ;;
  esac
done
[ -n "$IN" ] && [ -n "$PAT" ] || { echo "нужны ключи -i и -p" >&2; exit 1; }
OPTS=""
```

```
[ $SENS -eq 0 ] && OPTS="-i"
[ $NUMS -eq 1 ] && OPTS="$OPTS -n"
if [ -n "$OUT" ]; then
  grep $OPTS -- "$PAT" "$IN" > "$OUT"
  cat "$OUT"
else
  grep $OPTS -- "$PAT" "$IN"
fi
```

Проверка на файле `sample.txt` со строками `Begin`, `begin`, `BEGIN`, `end`:

```
$ ./findgrep.sh -i sample.txt -p begin
Begin
begin
BEGIN

$ ./findgrep.sh -i sample.txt -p Begin -C
Begin

$ ./findgrep.sh -i sample.txt -p begin -C -n -o out.txt
2:begin
```

Без ключа `-C` регистр не учитывается (используется `grep -i`), с ключом `-C` — учитывается; ключ `-n` добавляет номера строк, ключ `-o` направляет результат в файл. Результат показан на рис. 4.1.

```
vzhang@vzhang:~/work/os/lab11$ cat findgrep.sh
#!/bin/bash
# поиск строк по шаблону: -i файл, -o файл, -p шаблон, -C (учитывать регистр), -n (н
IN=""; OUT=""; PAT=""; SENS=0; NUMS=0
while getopts "i:o:p:Cnh" opt; do
  case $opt in
    i) IN="$OPTARG" ;;
    o) OUT="$OPTARG" ;;
    p) PAT="$OPTARG" ;;
    C) SENS=1 ;;
    n) NUMS=1 ;;
    *) echo "usage: $0 -i файл -p шаблон [-o файл] [-C] [-n]" >&2; exit 1 ;;
  esac
done
[ -n "$IN" ] && [ -n "$PAT" ] || { echo "нужны ключи -i и -p" >&2; exit 1; }
OPTS=""
[ $SENS -eq 0 ] && OPTS="-i"
[ $NUMS -eq 1 ] && OPTS="$OPTS -n"
if [ -n "$OUT" ]; then
  grep $OPTS -- "$PAT" "$IN" > "$OUT"
  cat "$OUT"
else
  grep $OPTS -- "$PAT" "$IN"
fi

vzhang@vzhang:~/work/os/lab11$ ./findgrep.sh -i sample.txt -p begin
Begin
begin
BEGIN

vzhang@vzhang:~/work/os/lab11$ ./findgrep.sh -i sample.txt -p Begin -C
Begin
```

Рисунок 4.1: Работа командного файла findgrep.sh

4.2 2. Программа на C и анализ кода возврата

Листинг `sign.c`:

```c
#include <stdio.h>

int main(void) {
    int n;
    if (scanf("%d", &n) != 1)
        return 2;
    if (n > 0) return 0;   /* больше нуля   */
    if (n < 0) return 1;   /* меньше нуля   */
    return 3;              /* равно нулю    */
```

```
}
```

Программа собрана командой `gcc -Wall -o sign sign.c`. Командный файл `checknum.sh` запускает её и анализирует `$?`:

```
#!/bin/bash
read -r -p "Введите число: " x
echo "$x" | ./sign
case $? in
  0) echo "Число $x больше нуля" ;;
  1) echo "Число $x меньше нуля" ;;
  3) echo "Число $x равно нулю" ;;
  *) echo "Ошибка ввода" ;;
esac
```

Результаты:

```
$ echo 15 | ./checknum.sh
Число 15 больше нуля
$ echo -3 | ./checknum.sh
Число -3 меньше нуля
$ echo 0 | ./checknum.sh
Число 0 равно нулю
```

Коды возврата проверены напрямую: для `42` — `0`, для `-7` — `1`, для `0` — `3`, что соответствует значениям, переданным в `exit(n)`. Результат показан на рис. 4.2.

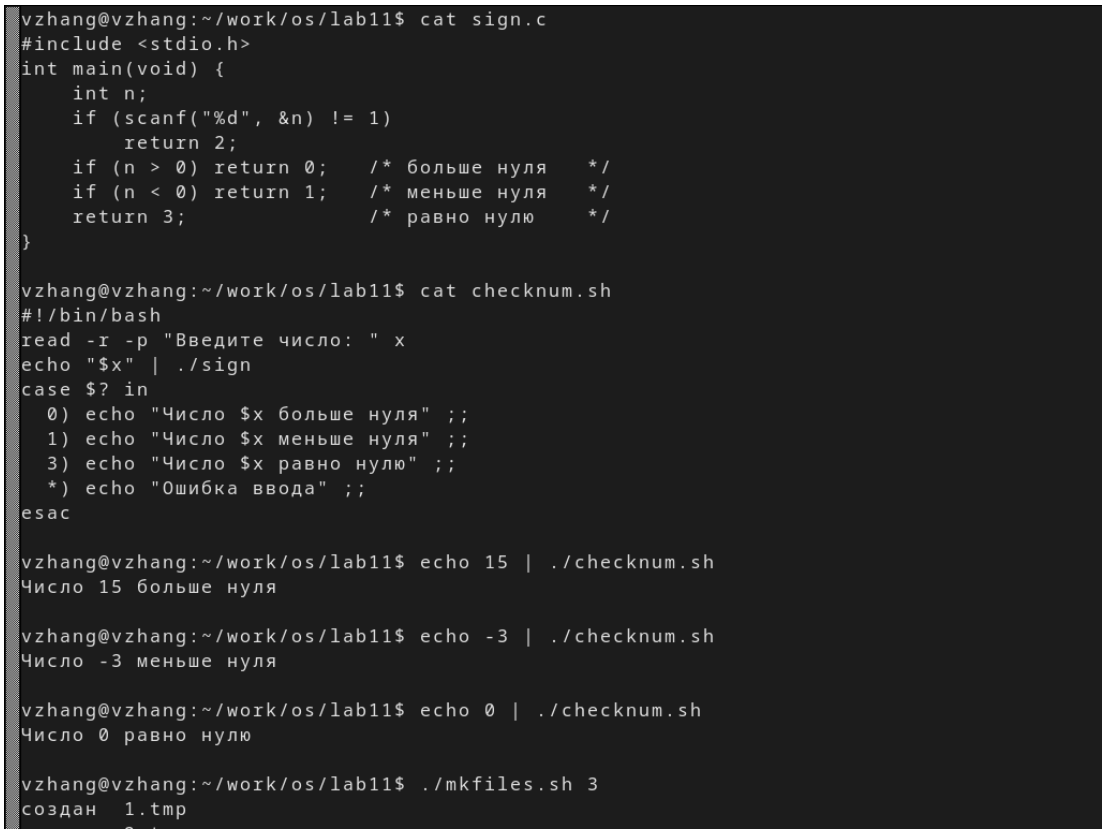


Рисунок 4.2: Программа sign.c, командный файл checknum.sh и анализ кода возврата

4.3 3. Создание и удаление N файлов

Листинг `mkfiles.sh`:

```
#!/bin/bash
# ./mkfiles.sh N        — создать файлы 1.tmp..N.tmp
# ./mkfiles.sh -d N     — удалить их
MODE=create
if [ "$1" = "-d" ]; then MODE=delete; shift; fi
N="${1:?укажите число файлов}"
n=1
while [ "$n" -le "$N" ]; do
```

```
  if [ "$MODE" = create ]; then
    touch "$n.tmp"; echo "создан  $n.tmp"
  else
    rm -f "$n.tmp"; echo "удалён  $n.tmp"
  fi
  n=$((n+1))
done
```

Работа:

```
$ ./mkfiles.sh 5
создан  1.tmp … создан  5.tmp
$ ./mkfiles.sh -d 5
удалён  1.tmp … удалён  5.tmp
```

Цикл `while` с инкрементом `n=$((n+1))` создаёт (или удаляет) файлы с именами `1.tmp`…`N.tmp`; число файлов передаётся аргументом командной строки.

4.4 4. Упаковка файлов каталога в архив

Листинг `tarlab.sh`:

```
#!/bin/bash
DIR="${1:?укажите каталог}"
ARCH="${2:-archive.tar.gz}"
DAYS="${3:-7}"
[ -d "$DIR" ] || { echo "не каталог: $DIR" >&2; exit 1; }
if [ "$DAYS" -gt 0 ]; then
  find "$DIR" -maxdepth 1 -type f -mtime -"$DAYS" -print0 | tar -czf "$ARCH" --null
  echo "упакованы файлы, изменённые менее $DAYS дней назад"
```

```
else
  find "$DIR" -maxdepth 1 -type f -print0 | tar -czf "$ARCH" --null -T -
  echo "упакованы все файлы"
fi
tar -tzf "$ARCH"
```

Результат:

```
$ ./tarlab.sh packdir pack.tar.gz 7
упакованы файлы, изменённые менее 7 дней назад
packdir/a.txt
packdir/b.txt
```

Команда `find` с опцией `-mtime -7` отбирает файлы, изменённые менее семи суток назад; `-print0` и `tar --null -T -` передают список файлов через стандартный ввод, что корректно работает с именами, содержащими пробелы.

5 Выводы

В ходе работы изучены логические управляющие конструкции и циклы оболочки bash: `if`, `case`, `while`, `until`, `for`, а также команды прерывания циклов `break` и `continue`. Освоена команда `getopts` для разбора ключей командной строки. Написаны и проверены четыре командных файла: скрипт поиска строк с ключами `-i`, `-o`, `-p`, `-C`, `-n` на основе `getopts` и `grep`; программа на языке C, возвращающая код через `exit(n)`, и командный файл, анализирующий `$?`; скрипт создания и удаления N пронумерованных файлов; скрипт упаковки файлов каталога в `tar` с отбором изменённых менее недели назад через `find -mtime -7`. Поставленные задачи выполнены полностью.

6 Ответы на контрольные вопросы

6.1 1. Предназначение команды getopts

`getopts` — встроенная команда оболочки для разбора позиционных параметров (ключей командной строки). Она последовательно перебирает ключи, заданные строкой-шаблоном (например, `"i:o:p:Cn"`), записывает очередной ключ в переменную `opt`, а его аргумент (если после буквы стоит `:`) — в `$OPTARG`; `$OPTIND` содержит индекс следующего необработанного аргумента, `$OPTARG` при ошибке указывает проблемную букву. Типичное применение — `while getopts … ; do case $opt in … esac; done`. Это избавляет от ручного разбора `$1`, `$2`, …

6.2 2. Какое отношение метасимволы имеют к генерации имён файлов?

Метасимволы (`*`, `?`, `[...]`, `{…}`) образуют шаблоны, по которым оболочка **генерирует** список имён файлов: перед выполнением команды она подставляет вместо шаблона все имена каталога, ему соответствующие (подстановка/globbing). Поэтому `rm *.tmp` удаляет все файлы с расширением `.tmp`, а `find`-подобные операции легко выражаются масками. Если ни одно имя не подходит, шаблон остаётся без изменений (если не включён `nullglob`).

6.3 3. Какие операторы управления действиями вы знаете?

`if … then … elif … else … fi` (ветвление), `case … in … esac` (выбор по шаблону), `for … in … do … done` (перебор), `for ((;;))` (C-подобный цикл), `while … do … done` (цикл с предусловием), `until … do … done` (цикл с постусловием), а также логические операторы `&&` (И) и `||` (ИЛИ), группировка `{ …; }` и `( … )`, оператор `!` (отрицание кода возврата). Для организации ветвления используются и команды `test`/`[ ]`.

6.4 4. Какие операторы используются для прерывания цикла?

`break` — немедленный выход из цикла (`break n` — из `n` вложенных циклов); `continue` — пропуск оставшихся команд текущей итерации и переход к следующей (`continue n` — для `n`-го уровня). Дополнительно цикл можно прервать командой `exit` (выход из скрипта) или сигналом.

6.5 5. Для чего нужны команды false и true?

Это встроенные команды, всегда возвращающие фиксированный код: `true` — 0 (успех/истина), `false` — 1 (неудача/ложь). Они используются как «заглушки» в условиях и для организации бесконечных циклов (`while true; do … done`), для проверки ветвления в скриптах, а также в конструкциях вида `if false; then …; fi`. В отличие от `/bin/true` и `/bin/false` они выполняются без запуска внешнего процесса.

6.6 6. Что означает строка if test -f man$s$/i.$s?

Она проверяет, существует ли **обычный файл** с путём, собранным из значений переменных: `man$s` — значение переменной `s`, приписанное к слову `man` (каталог вида `man1`, `man2`, ...), `$i` — значение переменной `i` (имя страницы), `.$s` — расширение, равное `s` (например, `man1/ls.1`). Условие истинно, если такой файл существует и является обычным файлом (`-f`). Такая строка типична для скрипта, перебирающего страницы руководства в каталогах `man1`, `man2`, ...

6.7 7. Объясните различия между конструкциями while и until

`while условие; do команды; done` выполняет тело, **пока** условие истинно (код возврата 0), то есть проверка — перед телом; `until условие; do команды; done` выполняет тело, **пока** условие ложно (до тех пор, пока условие не станет истинным), то есть это цикл с отрицанием условия. Обе конструкции — циклы с предусловием; если условие изначально соответственно ложно/истинно, тело не выполнится ни разу. `until` удобен, когда нужно ждать наступления события, а `while` — когда нужно повторять, пока оно не наступило (или пока условие сохраняется).

Список литературы

1. Robbins A. Bash Pocket Reference. O'Reilly Media, 2016. 156 c.